

Contrato de API

Loja Virtual VIP Imports

Endpoints, formatos de resposta e nomenclatura no backend e no frontend

Referência: tarefa 4 da Fatia 0 do Plano de Execução
Stack: Node.js + Fastify no backend, React + Vite no frontend
Base: `/api/v1` · **Formato:** JSON · **Codificação:** UTF-8

Por que este documento existe. Enquanto ele não estiver fechado, os dois devs de frontend não conseguem montar tela nenhuma — metade da capacidade da equipe fica parada. Ele é a peça que permite backend e frontend trabalharem em paralelo. Se algum formato aqui mudar depois que alguém já implementou, avise no grupo **antes** de mudar.

01 Convenções gerais

Regras que valem para todos os endpoints, sem exceção

1.1 Idioma

Rotas, campos e nomes de função em **português**, sem acento e em minúsculas. O banco já foi modelado em português (`produtos` , `marcas` , `selecao_itens`) — misturar inglês na API criaria tradução mental em toda camada. Exceções: palavras que não têm tradução usual em código (`slug` , `status` , `token`).

1.2 Formato de resposta

Toda resposta de sucesso devolve o dado diretamente. Listagens vêm dentro de um envelope com informação de paginação:

```
// item único
{ "id": 12, "codigo": "CHN-0042", "nome": "..." }

// listagem
{
  "dados": [ ... ],
  "paginacao": { "total": 11569, "pagina": 1, "porPagina": 24, "proximoCursor": "eyJpZCI6MjR9" }
}
```

1.3 Formato de erro

Sempre o mesmo formato, em qualquer erro, para o frontend tratar em um lugar só:

```
{
  "erro": {
    "codigo": "PRODUTO_NAO_ENCONTRADO",
    "mensagem": "Produto não encontrado.",
    "campos": { "email": "Formato inválido." }    // opcional, só em validação
  }
}
```

`mensagem` é o texto que o frontend pode exibir direto ao visitante. `codigo` é para o frontend decidir o que fazer. **Nunca vazar detalhe técnico** (nome de tabela, stack trace, caminho de arquivo) nesses campos.

1.4 Códigos de status

Código	Quando	Observação
200	Sucesso	Leitura, atualização e exclusão bem-sucedidas
201	Criado	Só em POST que cria registro. Devolve o objeto criado
400	Dados inválidos	Preencher <code>erro.campos</code> indicando o que falhou
401	Não identificado	Falta sessão. O frontend redireciona para identificação
403	Sem permissão	Está identificado, mas não pode. Ex: cliente tentando rota de admin
404	Não encontrado	
409	Conflito	Ex: código de produto já existente
429	Excesso de requisições	Limite atingido na identificação
500	Erro do servidor	Mensagem genérica ao visitante, detalhe só no log

1.5 Paginação

Dois modos, e a escolha não é livre:

- Vitrine (público):** paginação por cursor, usando `?cursor=` . É o que foi decidido na modelagem — `OFFSET` profundo abandona o índice e cai em varredura completa com 11 mil produtos.
- Painel administrativo:** `?pagina=` e `?porPagina=` são aceitáveis, porque o admin não navega até a página 300.

1.6 Sessão

Cliente e administrador têm sessões **separadas**, com cookies diferentes. Ambos `httpOnly` , `secure` e `sameSite=lax` . O frontend nunca lê o token: envia as requisições com `credentials: "include"` e o navegador cuida do resto.

Regra de segurança: nenhuma rota `/admin/*` pode ser acessível com sessão de cliente. A verificação é aplicada no grupo inteiro de rotas, nunca uma a uma.

02 Endpoints públicos — catálogo

Fatias 1 e 2 - não exigem identificação

Rota	Descrição	Backend	Frontend
<code>GET</code> /home	Tudo da página inicial em uma chamada	montarHome home.routes.ts	buscarHome() useHome()
<code>GET</code> /produtos	Listagem com filtros, busca e cursor	listarProdutos produtos.routes.ts	listarProdutos() useProdutos()
<code>GET</code> /produtos/:codigo	Detalhe do produto com imagens e variações	obterProduto	obterProduto() useProduto()
<code>GET</code> /produtos/:codigo/relacionados	Até 8 itens da mesma marca ou categoria	listarRelacionados	listarRelacionados()
<code>GET</code> /marcas	18 marcas com contagem de produtos	listarMarcas marcas.routes.ts	listarMarcas() useMarcas()
<code>GET</code> /colecões	Feminino e Masculino	listarColecões colecões.routes.ts	listarColecões()
<code>GET</code> /colecões/:slug/categorias	Categorias daquela coleção	listarCategoriasDaColecao	listarCategorias() useCategorias()

Atenção na rota de categoria. O slug de categoria é único **por coleção**, não global — "bolsas" existe em Feminino e em Masculino. Por isso a URL do site é `/feminino/bolsas` e a rota da API sempre carrega a coleção. Não existe `GET /categorias/:slug` sozinho.

Parâmetros de `GET /produtos`

Parâmetro	Tipo	Descrição
colecão	slug	feminino ou masculino
categoria	slug	Exige coleção junto, pelo motivo acima
marca	slug	Aceita múltiplos separados por vírgula
busca	texto	Procura em nome, código e marca
ordem	enum	recentes (padrão) ou nome
cursor	texto	Devolvido na resposta anterior. Ausente = primeira página
porPágina	número	Padrão 24, máximo 48

Produtos com status `oculto` nunca aparecem. Produtos `esgotado` aparecem com a marcação, porque servem de vitrine.

Resposta de `GET /produtos`

```
{
  "dados": [
    {
      "id": 812,
      "codigo": "CHN-0042",
      "nome": "Bolsa acolchoada com corrente",
      "status": "normal",           // normal | esgotado
      "destaque": false,
      "marca": { "nome": "Chanel", "slug": "chanel" },
      "categoria": { "nome": "Bolsas", "slug": "bolsas" },
      "colecão": { "nome": "Feminina", "slug": "feminino" },
      "capa": { "url": "https://...", "alt": "Bolsa Chanel preta" }
    }
  ],
  "paginacao": { "total": 11569, "porPágina": 24, "proximoCursor": "eyJpZCI6ODUyYfQ" }
}
```

Sem preço. Nenhuma resposta da API devolve valor, e o banco não tem esse campo. O frontend exibe "Valor confirmado pelo atendimento" como texto fixo, não vindo da API.

Resposta de `GET /produtos/:codigo`

```
{
  "id": 812,
  "codigo": "CHN-0042",
  "nome": "Bolsa acolchoada com corrente",
  "descricao": "Couro legítimo, ferragem dourada...",
  "status": "normal",
  "marca": { "nome": "Chanel", "slug": "chanel" },
  "categoria": { "nome": "Bolsas", "slug": "bolsas" },
  "colecão": { "nome": "Feminina", "slug": "feminino" },
  "imagens": [
    { "id": 3301, "url": "https://...", "alt": "...", "ordem": 1 }
  ],
  "variacoes": [
    { "id": 55, "tipo": "tamanho", "valor": "M", "disponivel": true },
    { "id": 56, "tipo": "cor", "valor": "Preto", "disponivel": false }
  ]
}
```

03 Endpoints — área do cliente

Fatia 3 - identificação por e-mail, sem senha

Rota	Descrição	Backend	Frontend
POST /clientes/identificar	Informa o e-mail; cria a conta se não existir	identificarCliente clientes.routes.ts	identificar() useSessaoCliente()
GET /clientes/eu	Dados do cliente da sessão atual	obterClienteAtual	obterClienteAtual()
PATCH /clientes/eu	Edita nome e telefone	atualizarClienteAtual	atualizarCliente()
POST /clientes/sair	Encerra a sessão	encerrarSessaoCliente	sair()
GET /favoritos	Produtos favoritos	listarFavoritos favoritos.routes.ts	listarFavoritos() useFavoritos()
POST /favoritos	Adiciona um favorito	adicionarFavorito	adicionarFavorito()
DEL /favoritos/:produtoId	Remove um favorito	removerFavorito	removerFavorito()
GET /carrinho	Itens do carrinho	obterCarrinho carrinho.routes.ts	obterCarrinho() useCarrinho()
POST /carrinho	Adiciona item com variação	adicionarAoCarrinho	adicionarAoCarrinho()
PATCH /carrinho/:itemId	Troca a variação escolhida	atualizarItemCarrinho	atualizarItem()
DEL /carrinho/:itemId	Remove item	removerItemCarrinho	removerItem()
POST /carrinho/migrar	Move o carrinho do navegador para a conta	migrarCarrinhoAnonimo	migrarCarrinho()
POST /selecoes	Grava a seleção e devolve o texto do WhatsApp	criarSelecao selecoes.routes.ts	enviarSelecao()
GET /selecoes	Histórico de seleções enviadas	listarSelecoesDoCliente	listarSelecoes() useSelecoes()

POST /carrinho/migrar é obrigatório e fácil de esquecer. Quem não se identificou monta o carrinho no `localStorage`. No momento em que informa o e-mail, esses itens precisam ir para a conta. Se o carrinho esvaziar na identificação, perde-se a venda — é o erro mais comum desse fluxo.

POST /selecoes — requisição e resposta

```
// requisição – corpo vazio: usa o carrinho atual da sessão
POST /api/v1/selecoes

// resposta 201
{
  "id": 44,
  "criadoEm": "2026-09-02T14:31:00Z",
  "itens": [
    { "codigo": "CHN-0042", "nome": "Bolsa acolchoada", "marca": "Chanel", "variacao": "M / Preto" }
  ],
  "mensagemWhatsapp": "Olá! Tenho interesse nestes itens:\n\n• CHN-0042 – Bolsa acolchoada (Chanel, M / Preto)\n\nEnviado pelo site.",
  "linkWhatsapp": "https://wa.me/5541984975960?text=01%C3%A1%21%20Tenho..."
}
```

O backend monta o link, não o frontend. Assim o número da loja e o formato da mensagem ficam em um lugar só. O frontend apenas abre `linkWhatsapp` em nova aba. E lembre: isso não usa a API do WhatsApp nem tem custo.

04 Endpoints — painel administrativo

Fatia 4 - todas exigem sessão de administrador

4.1 Sessão e visão geral

Rota	Descrição	Backend	Frontend
POST /admin/sessao	Login com e-mail e senha	autenticarAdmin admin/sessao.routes.ts	entrarAdmin() useSessaoAdmin()
DEL /admin/sessao	Logout	encerrarSessaoAdmin	sairAdmin()
GET /admin/resumo	Números da tela inicial	obterResumo admin/resumo.routes.ts	obterResumo() useResumo()

4.2 Produtos

Rota	Descrição	Backend	Frontend
GET /admin/produtos	Listagem com busca e filtros, incluindo ocultos	listarProdutosAdmin admin/produtos.routes.ts	listarProdutosAdmin() useProdutosAdmin()
GET /admin/produtos/:id	Produto para edição	obterProdutoAdmin	obterProdutoAdmin()
POST /admin/produtos	Cria produto	criarProduto	criarProduto()
PATCH /admin/produtos/:id	Edita produto	atualizarProduto	atualizarProduto()
DEL /admin/produtos/:id	Exclui produto	excluirProduto	excluirProduto()
POST /admin/produtos/:id/duplicar	Copia o produto como base para outro	duplicarProduto	duplicarProduto()
PATCH /admin/produtos/lote	Aplica mudança a vários de uma vez	atualizarProdutosEmLote	atualizarEmLote()
POST /admin/produtos/:id/imagens	Adiciona imagens por URL	adicionarImagens admin/imagens.routes.ts	adicionarImagens()
PATCH /admin/produtos/:id/imagens/ordem	Reordena as fotos	reordenarImagens	reordenarImagens()
DEL /admin/imagens/:id	Remove uma imagem	excluirImagem	excluirImagem()
PATCH /admin/produtos/:id/variacoes	Substitui as variações do produto	definirVariacoes	definirVariacoes()

Imagens são por URL, não por upload. Nesta fase não hospedamos arquivo: o corpo de `POST /admin/produtos/:id/imagens` recebe uma lista de URLs externas. Quando passarmos a hospedar, a rota ganha upload sem mudar de nome.

4.3 Taxonomia, banners e destaques

Rota	Descrição	Backend	Frontend
GET POST /admin/marcas PATCH DEL /admin/marcas/:id	CRUD de marcas	listarMarcasAdmin criarMarca atualizarMarca excluirMarca	useMarcasAdmin()
GET POST /admin/categorias PATCH DEL /admin/categorias/:id	CRUD de categorias	listarCategoriasAdmin criarCategoria atualizarCategoria excluirCategoria	useCategoriasAdmin()
GET POST /admin/banners PATCH DEL /admin/banners/:id	CRUD do carrossel da home	listarBanners criarBanner atualizarBanner excluirBanner	useBanners()
PATCH /admin/destaques/produtos	Define e ordena produtos da home	definirProdutosDestaque admin/destaques.routes.ts	definirDestaques() useDestaques()
PATCH /admin/destaques/categorias	Define e ordena categorias da home	definirCategoriasDestaque	definirDestaquesCategorias()

Excluir marca ou categoria com produtos vinculados deve devolver 409, com a contagem de produtos afetados na mensagem. Nunca deixar produto órfão.

4.4 Seleções e clientes

Rota	Descrição	Backend	Frontend
GET /admin/selecoes	Seleções recebidas, mais recentes primeiro	listarSelecoesAdmin admin/selecoes.routes.ts	listarSelecoesAdmin() useSelecoesAdmin()
GET /admin/selecoes/:id	Detalhe com os itens congelados	obterSelecaoAdmin	obterSelecao()
GET /admin/clientes	Clientes cadastrados	listarClientesAdmin admin/clientes.routes.ts	listarClientes() useClientesAdmin()

Seleção mostra o dado congelado. `selecao_item` guarda código, nome, marca e categoria do momento do envio. Se o produto foi renomeado ou apagado depois, o painel continua mostrando o que o cliente realmente viu.

05 Controle de acesso — previsto, não implementado

Modelado no banco, congelado até decisão do cliente

O banco já suporta três modos de acesso. **Nenhuma dessas rotas deve ser construída agora** — elas estão aqui para o frontend saber onde encaixariam e não desenhar arquitetura que as impeça.

Rota	Descrição	Backend	Modo
GET /acesso/estado	Diz ao frontend se pode navegar e qual tela mostrar	obterEstadoAcesso	todos
POST /acesso/senha	Valida a senha compartilhada	validarSenhaAcesso	2
POST /acesso/solicitar	Pede aprovação informando o e-mail	solicitarAcesso	3
GET /admin/acesso/fila	Pendentes de aprovação	listarFilaAprovacao	3
PATCH /admin/acesso/:clienteId	Aprova, recusa ou revoga	decidirAcesso	3
PATCH /admin/configuracao/acesso	Troca o modo ativo	definirModoAcesso	todos

Consequência para o frontend hoje: centralize as chamadas em uma camada só (ver seção 07). Se cada tela chamar `fetch` direto, ligar o controle de acesso depois exige mexer em todas elas. Com a camada centralizada, é um tratamento de **401** em um lugar.

06 Nomenclatura no backend

Onde cada coisa mora e como se chama

apps/api/src/			
├─ rotas/			
│ ├─ home.routes.ts	produtos.routes.ts	marcas.routes.ts	
│ ├─ colecoes.routes.ts	clientes.routes.ts	favoritos.routes.ts	
│ └─ carrinho.routes.ts	selecoes.routes.ts		
├─ admin/			
│ ├─ sessao.routes.ts	produtos.routes.ts	imagens.routes.ts	
│ ├─ marcas.routes.ts	categorias.routes.ts	banners.routes.ts	
│ ├─ destaques.routes.ts	selecoes.routes.ts	clientes.routes.ts	
│ └─ resumo.routes.ts			
├─ servicos/	produtos.service.ts, carrinho.service.ts ...		
├─ esquemas/	produtos.schema.ts (validação de entrada e saída)		
├─ middlewares/	exigirCliente.ts, exigirAdmin.ts, limitarTaxa.ts		
└─ erros/	AppError.ts, codigos.ts		

Elemento	Padrão	Exemplo
Arquivo de rota	substantivo plural	produtos.routes.ts
Handler	verbo + substantivo	listarProdutos, criarSelecao
Handler do admin	sufixo Admin quando existe versão pública	listarProdutosAdmin
Serviço	<domínio>.service.ts	produtosService.listar()
Rota na URL	plural, minúsculo, sem acento	/selecoes, não /seleções
Campo JSON	camelCase	criadoEm, proximoCursor
Coluna no banco	snake_case	criado_em — a conversão é do serviço
Código de erro	MAIÚSCULO_COM_UNDERSCORE	PRODUTO_NAO_ENCONTRADO

Verbos padronizados, para ninguém inventar sinônimo: `listar` (coleção), `obter` (um item), `criar`, `atualizar`, `excluir`, `definir` (substitui o conjunto todo).

07 Nomenclatura no frontend

Camada de API centralizada, tipos e hooks

apps/web/src/	
└─ api/	
└─ cliente.ts	wrapper do fetch: base URL, credentials, tratamento de erro
└─ produtos.ts	marcas.ts colecoes.ts home.ts
└─ sessao.ts	favoritos.ts carrinho.ts selecoes.ts
└─ admin/	produtos.ts marcas.ts destaques.ts ...
└─ hooks/	useProdutos.ts useCarrinho.ts useSessaoCliente.ts ...
└─ tipos/	produto.ts cliente.ts selecao.ts
└─ componentes/	CardProduto.tsx BarraFiltros.tsx CarrinhoLateral.tsx

Regra que não se quebra: nenhuma tela chama `fetch` diretamente. Tudo passa por `api/cliente.ts`. É ele que aplica a base URL, envia `credentials`: `"include"`, converte o formato de erro e trata **401** globalmente. Sem isso, ligar o controle de acesso depois vira mexer em todas as telas.

Elemento	Padrão	Exemplo
Função de API	mesmo verbo do backend	<code>listarProdutos()</code> , <code>obterProduto()</code>
Hook	<code>use</code> + substantivo	<code>useProdutos()</code> , <code>useCarrinho()</code>
Hook do admin	sufixo <code>Admin</code>	<code>useProdutosAdmin()</code>
Tipo	PascalCase, singular	<code>Produto</code> , <code>ItemCarrinho</code>
Tipo de resposta paginada	<code>Pagina<T></code>	<code>Pagina<Produto></code>
Componente	PascalCase, em português	<code>CardProduto.tsx</code>

Tipos compartilhados

```
export type StatusProduto = "normal" | "esgotado" | "oculto";

export interface Referencia { nome: string; slug: string; }

export interface Produto {
  id: number;
  codigo: string;
  nome: string;
  descricao?: string;
  status: StatusProduto;
  destaque: boolean;
  marca: Referencia;
  categoria: Referencia;
  colecao: Referencia;
  capa?: Imagem;
  imagens?: Imagem[];
  variacoes?: Variacao[];
}

export interface Pagina<T> {
  dados: T[];
  paginacao: { total: number; porPagina: number; proximoCursor?: string };
}
```

Onde os tipos moram. Coloque em `packages/tipos`, importado pelos dois lados. Assim, mudar um campo no backend quebra o build do frontend imediatamente, em vez de virar `undefined` em produção.

08 Ordem de implementação

O que o backend entrega primeiro para não travar ninguém

Ordem	Endpoints	Destrava
1	<code>GET /produtos</code> , <code>/produtos/:codigo</code> , <code>/marcas</code> , <code>/colecoes</code>	Todas as telas de catálogo (Fatia 1)
2	<code>GET /home</code>	Página inicial (Fatia 2)
3	<code>/clientes/*</code> , <code>/carrinho/*</code> , <code>/selecoes</code>	Área do cliente e carrinho (Fatia 3)
4	<code>/admin/sessao</code> , <code>/admin/produtos/*</code>	Painel — a maior parte das telas (Fatia 4)
5	Demais rotas <code>/admin/*</code>	Telas de taxonomia, destaques e consultas

Enquanto um endpoint não existe, o frontend consome os dados falsos da tarefa 8, no formato exato deste documento. Trocar depois é mudar a origem em `api/cliente.ts` — nada mais.